

BenchCPU: Performance Distribution-Aware CPU Benchmarking over Open Configuration Spaces

Anonymous Author(s)

Abstract

For decades, industry-standard CPU benchmarks like SPEC CPU have relied on a *static workload configurations* paradigm—pairing diverse workloads with fixed configurations (e.g., dataset, concurrency) to report an aggregated performance score. However, in the era of heterogeneous multi-core architectures and rapidly evolving applications, this rigid paradigm increasingly devolves into a misleading “*configuration game*.” Our extensive exploration of the SPEC CPU configuration space reveals a critical flaw: fixed configurations mask an average of 20% performance gap compared to optimal space-explored configurations. More alarmingly, these static workload configuration evaluations often suffer from *biased evaluation results*, frequently manifesting as *outliers* that starkly deviate from the CPU’s true performance distribution.

To fundamentally bridge this methodology gap, we propose *BenchCPU*, an annually-updated, *performance distribution-aware* benchmarking that shifts CPU evaluation from static workload configurations to open configuration spaces. This open configuration space framework represents a paradigm shift: from solely considering the diversity of workloads to considering the intertwined situation of multiple dimensions (including but not limited to workload, dataset, and concurrency). To ensure fair and cost-effective comparisons, BenchCPU defines a calibrated reference space reflecting real-world deployments. By employing randomized sampling and momentum filtering across this space, it dynamically selects statistically continuous configuration samples for CPU evaluation.

Extensive evaluations demonstrate that BenchCPU successfully eliminates the biased evaluation result dilemma inherent in legacy benchmarks. Principal Component Analysis (PCA) reveals that a targeted subset of just 5 representative BenchCPU configurations achieves competitive microarchitectural coverage against the entire suite of 43 statically-configured SPEC CPU2017 workloads on a typical Intel Xeon Platform. By enabling extensive configuration space exploration, BenchCPU exposes novel microarchitectural insights that remain completely invisible to traditional static workload configurations based benchmarks.

CCS Concepts

• General and reference → Cross-computing tools and techniques; Cross-computing tools and techniques.

Keywords

CPU Benchmarking, Performance Evaluation, Microarchitecture, Parallel Computing

ACM Reference Format:

Anonymous Author(s). 2026. BenchCPU: Performance Distribution-Aware CPU Benchmarking over Open Configuration Spaces. In *Proceedings of*

International Conference on Parallel Architectures and Compilation Techniques (PACT '26). ACM, New York, NY, USA, 11 pages.

1 Introduction

Rigorous CPU evaluation is the bedrock of modern processor development, serving as the critical basis for architectural decisions, performance tuning, and cross-generation comparisons. For decades, industry-standard benchmark suites, notably SPEC CPU [34], have established a *static workload configurations* paradigm [13, 14, 29]. Formally, this paradigm conceptualizes CPU benchmarking as measuring performance along a few predefined, static workload configurations—pairing representative workloads with a fixed dataset and a specific concurrency level [17, 18]. A CPU’s overall performance score [33] is aggregated within these highly constrained workload configurations. However, as processor architectures evolve toward highly complex, non-linear, and heterogeneous multi-core designs, their performance becomes profoundly sensitive to evaluation configurations. Real-world demands now constitute a massive, multi-dimensional continuum spanned by diverse workloads, heterogeneous input datasets, and a combinatorial explosion of compilation and concurrency configurations [2, 23, 25]. In this modern context, evaluating a processor based on a few predefined, static workload configurations has its inherent limitations.

Our extensive characterization of existing CPU benchmarking practices reveals the inherent limitations of the traditional static workload configurations paradigm, manifesting in three critical crises: **(1) Configuration Game:** By fixing benchmark configurations, existing suites lack the capability to detect or prevent vendor over-optimization. Our analysis shows that configuration nuances of the same workload can significantly affect CPU performance comparison results and even cause **rank inversion**. **(2) Biased Evaluation Results:** Existing benchmarking results may manifest as statistically unrepresentative **outliers** that starkly deviate from the CPU’s true performance distribution, making longitudinal tracking and cross-architecture comparisons highly unreliable. **(3) Architectural Blind Spots:** The static workload configurations hide the true capability boundaries of modern CPUs. Our space-exploration experiments reveal an average of over **20% performance gap** between the SPEC-recommended configurations and the optimal configurations found within the broader space.

To bridge this critical methodology gap, we propose **BenchCPU**, a novel benchmarking paradigm that shifts CPU evaluation from static workload configurations based evaluation to statistically robust, open configuration space driven distributions. As shown in Fig. 1, BenchCPU fundamentally dismantles the monolithic structure of traditional benchmarks by decoupling the evaluation space into three orthogonal, highly extensible dimensions: *workload, dataset, and concurrency level*. Crucially, BenchCPU is not restricted to these initial axes; for example, we further expand the evaluation

PACT '26, Chicago, Illinois, USA
2026.

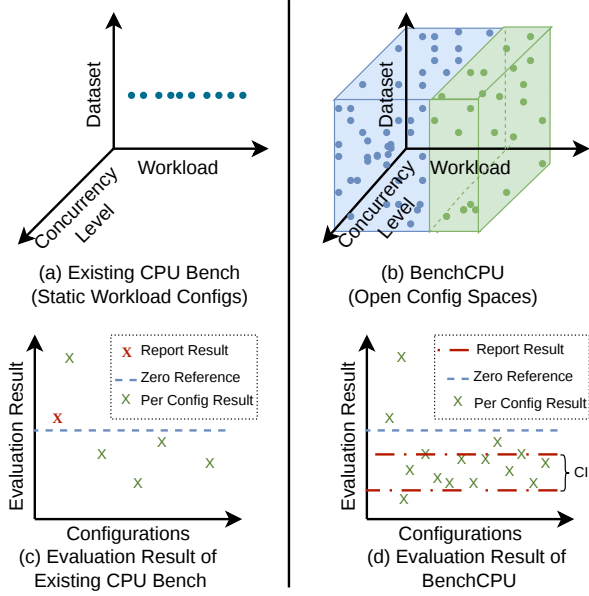


Figure 1: Comparison between BenchCPU and existing CPU benchmarks: from static workload configurations based evaluation to evaluating CPUs in open configuration spaces spanning workload, dataset, and concurrency level, thereby characterizing CPU performance as a distribution (confidence intervals, CI) rather than an aggregated score.

space by exposing compiler flags (e.g., -O1, -O2, -O3) as an additional dimension for C/C++ workloads.

To operationalize this, BenchCPU eliminates static constraints by transforming workloads, datasets, and concurrency levels into dynamically explorable configuration parameters. Then, BenchCPU evaluates CPUs across an open configuration space, preventing biased evaluation results and exposing true microarchitectural boundaries. Furthermore, to ensure this open configuration space evaluation remains both historically consistent and forward-looking, BenchCPU structures its workload dimension into two distinct tiers, Basic Workloads serve as a stable baseline anchor and Emerging Workloads track the bleeding-edge demands. Traditional benchmarks update infrequently, forcing a hard choice: keep outdated workloads for historical comparisons, or adopt new ones to stay relevant. BenchCPU resolves this by establishing a highly stable *Basic Space* (anchored workloads ensuring long-term evaluation continuity) alongside a dynamically evolving *Emerging Space* (updated annually to aggressively track bleeding-edge demands).

Operating within these expansive spaces, BenchCPU fundamentally transforms CPU benchmarking from a static configuration measurement into a rigorous *stochastic inference process*. To navigate the combinatorial explosion of configurations without incurring prohibitive execution costs, BenchCPU employs a statistically disciplined evaluation methodology. Rather than exhaustive execution, it dynamically constructs statistically continuous evaluation sets via stratified random sampling [26, 36] across both the Basic and Emerging Spaces. To maintain cross-version stability and

mitigate transient evaluation noise, this sampling is strictly regulated by a *momentum-based filter*. Grounded in the Central Limit Theorem (CLT) [9, 20], this integrated mechanism ensures that the aggregated performance estimations inherently converge to a normal distribution, formally modeled as $\mathcal{N}(\mu, \sigma^2/n)$. This mathematical foundation provides a powerful probabilistic guarantee: it effectively suppresses the distributional drift caused by generational workload updates and configuration anomalies, thereby ensuring strict longitudinal consistency over time. Consequently, BenchCPU delivers a level of statistical robustness and architectural fidelity that legacy static benchmarks inherently lack.

Our primary contributions are summarized as follows:

- (1) **Methodological Paradigm Shift:** We identify and quantify the severe *inherent limitations* in legacy CPU benchmarks. We pioneer a methodology shift from static workload configurations evaluation to performance distribution-aware benchmarking over open configuration spaces, fundamentally solving these limitations.
- (2) **The BenchCPU Framework:** We design and open-source BenchCPU, a performance distribution-aware benchmarking framework that constructs open configuration spaces by transforming datasets and concurrency levels into dynamic parameters, intersected with our partitioned *Basic* and *Emerging* workloads. To navigate the combinatorial explosion, BenchCPU employs stratified random sampling and a momentum-based filter, guaranteeing cost-effective, mathematically rigorous, and longitudinally consistent performance distributions without exhaustive execution.
- (3) **Compelling Results & Architectural Insights:** Extensive evaluations demonstrate BenchCPU's superiority. Utilizing Principal Component Analysis (PCA), we prove that a targeted subset of just 5 *representative BenchCPU configurations* achieves competitive microarchitectural coverage against all 43 *statically-configured SPEC CPU2017 workloads* on a typical Intel Xeon Platform. Furthermore, BenchCPU's dataset-dimension exploration exposes novel architectural insights: on two server CPUs, dense input-scale sweeping surfaces a clear *runtime-bounded vs. data-bounded* workload taxonomy and attributes a counter-intuitive 3.25 $\times$ L2-over-L1D MPKI inversion in cassandra workload into microarchitecture mechanisms. This is a diagnosis unattainable from any static workload configurations based benchmark.

The remainder of the paper is organized as follows. Section 2 introduces related work. Section 3 discusses the crises with existing CPU benchmark suites and shows our motivation. Section 4 provides the methodology of BenchCPU. Section 5 evaluates BenchCPU, and Section 6 concludes.

2 Related Work

This section reviews the literature across three dimensions critical to CPU performance evaluation: state-of-the-practice benchmark suites, workload characterization researches, and the formal methodology of computer evaluation. Throughout this review, we highlight the systemic gaps that BenchCPU is designed to bridge.

2.1 State-of-the-Practice CPU Benchmark Suites and Configuration Game

For decades, monolithic benchmark suites have been the primary vehicles for driving architectural research. SPEC CPU [34] stands as the industry standard for compute-intensive performance, providing comparative measurements through metrics like SPECspeed and SPECrate across various hardware platforms [28]. Similarly, PARSEC [4, 46] was pioneered to represent emerging multi-threaded workloads for shared-memory chip-multiprocessors (CMPs). While benchmarks strive for standardizing measurement, researchers have long recognized that extraneous system configurations severely impact performance measurement. Hennessy et al. [13] demonstrated that software-stack shifts (e.g., matrix multiplication algorithms) can induce performance gaps spanning orders of magnitude. Mytkowicz et al. [27] exposed how innocuous environmental changes (e.g., link order, environment variables) cause pervasive measurement biases in architectural evaluation. Wang et al. [38], and Benson et al. [2] highlighted how experimental configurations dictate overarching performance conclusions. These findings highlight the *static workload configurations* paradigm of existing CPU benchmark suites has an inherent limitation of "*configuration game*." In contrast to these static snapshots, BenchCPU fundamentally decouples the evaluation space into workload, dataset, and concurrency level, shifting the paradigm towards a continuous, distribution-aware *open configuration space evaluation*.

2.2 Workload Characterization and Space Exploration

Workload characterization is essential for understanding the interaction between software and underlying CPU architectures. A wealth of prior research has profiled workloads at various abstraction levels. At the microarchitectural level, hardware performance counters are widely utilized to capture execution bottlenecks [8, 28, 41]. At higher abstraction levels, researchers analyze Instruction Set Architecture (ISA) traces [3, 7, 16–19, 40] and intermediate representations (IR)[5, 32] to achieve microarchitecture-independent and ISA-independent characterizations. Recent efforts, such as the whole-picture characterization (WPC)[37], attempt to integrate metrics across the system stack. Furthermore, several studies have employed Principal Component Analysis (PCA) and clustering algorithms on SPEC CPU to validate benchmark coverage and redundancy [25, 28, 30, 31]. For instance, Panda et al. [28] analyzed the clustering of EDA and graph applications to confirm SPEC CPU2017's diversity. The critical limitation of these characterization studies is that they are deeply constrained by the *static workload configurations* of the underlying benchmarks. Profiling a SPEC workload using its recommended configuration would yield a *biased* architectural footprint, even masking the 20% performance variance and bottleneck shifts that occur when datasets or concurrency levels change. BenchCPU leverages these proven characterization techniques (such as PCA) but applies them across our *open configuration spaces*. This enables us to dynamically identify a representative 5-configuration subset that rivals the coverage of the entire 43-workload static SPEC suite on a typical Intel Xeon Platform, exposing microarchitectural insights previously *hidden* by static profiling.

2.3 Evaluation Methodology and Statistical Rigor

Achieving fairness and consistency in computer evaluation requires rigorous theoretical foundations. Zhan et al. [42–45] proposed *Evaluatology* as the science of uncovering the effects, defining *evaluation* as conducting deliberate experiments where well-defined conditions are applied to specific subjects. Building on this, Wang et al. [36] extended Evaluatology to computer systems, defining key components and formalizing how specific system components (e.g., compilers, OS) attribute to overall performance results. While Evaluatology provides a robust epistemological and theoretical framework, there is a distinct lack of practical, scalable benchmark suites that *instantiate* these principles for modern CPU evaluation. Theoretical definitions of "evaluation condition" should be translated into actionable methodologies to combat biased evaluation results and longitudinal drift. BenchCPU bridges this *theoretical-practical* divide. By incorporating evaluatology principles with statistical theorems (e.g., Central Limit Theorem) via stratified random sampling and momentum filtering, BenchCPU transforms theoretical evaluation science into an automated, mathematically rigorous, and space-explored benchmarking paradigm.

3 Motivation: The Crises of Static Workload Configurations Based Evaluation

Existing CPU benchmark suites implicitly assume that a rigidly defined "static workload configurations" can accurately proxy a processor's holistic performance capability. To empirically test this hypothesis, we conduct a deep characterization of the industry-standard SPEC CPU2017 suite. By fundamentally unlocking its static evaluation boundaries, we reveal three critical methodological crises that plague modern CPU evaluation: *The Configuration Game*, *Biased Evaluation Results*, and *Architectural Blind Spots*.

3.1 Unlocking the SPEC CPU2017 Configuration Space

To expose the hidden performance dynamics, we first constructed a methodology to "unbox" the static SPEC CPU2017 benchmark. Instead of relying solely on the official test, train, and ref datasets, we reverse-engineered SPEC's official dataset generation paradigms. Consequently, we successfully generated over 30 valid, high-fidelity datasets of continuously varying sizes for each SPEC workload. Furthermore, we expanded the execution concurrency level to a continuous range of [1,128] (up to twice the maximum logical cores of the evaluated CPUs in Table 1), and opened the compiler flag (e.g., -O1, -O2, -O3).

We open-sourced this combinatorial configuration generation toolset as a sub-project of BenchCPU (note that users must provide their own SPEC CPU2017 license to run the workloads). Armed with this toolset, we systematically sampled the configuration space to evaluate CPU A (an x86 processor) and CPU B (an ARM processor) (Hardware details in Table 1).

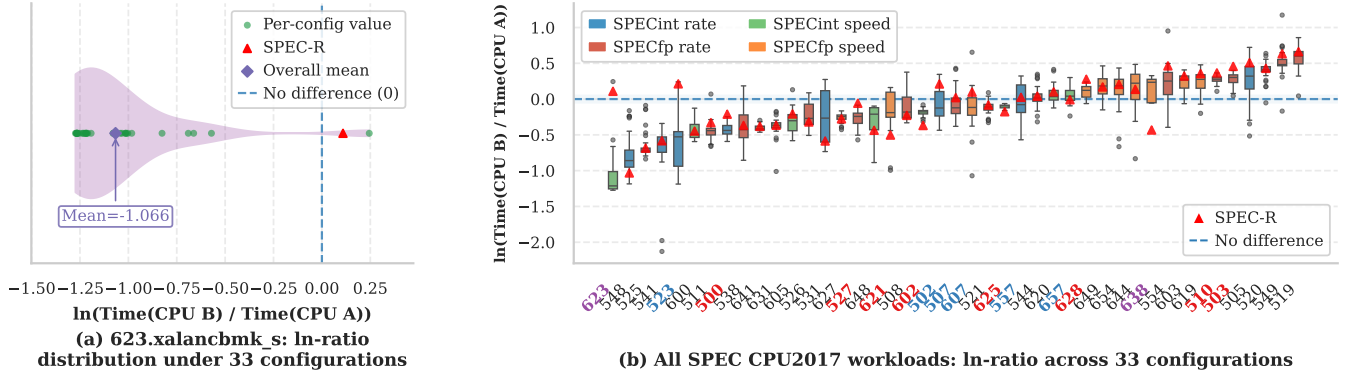


Figure 2: Execution-time-based comparisons between CPU A and CPU B can lead to opposite conclusions under different configurations. Panel (a) shows results for 623.xalancbm_s across 33 configurations, and panel (b) summarizes all SPEC CPU2017 workloads across the 33 configurations. In panel (b), x-axis workload labels are color-coded: red indicates SPEC-R (SPEC-recommended configuration evaluation) is an outlier with respect to the boxplot whiskers, blue indicates the SPEC-R sign is opposite to the mean sign of the 32 non-SPEC-R configurations, and purple indicates both conditions. For each box in (b), the bottom and top edges denote Q1 and Q3, the center line denotes the median, whiskers extend to the most extreme non-outlier values within $1.5 \times \text{IQR}$, and points beyond whiskers are marked [39].

Table 1: Machine information in our experiments.

Machine	CPU A	CPU B
Architecture	x86	ARM
Socket(s)	2	2
Core(s) Per Socket	14	32
Thread(s) Per Core	2	1
L1 (Per Core)	32KiB I, 32KiB D	64KiB I, 64KiB D
L2 (Per Core)	1MiB	512KiB
L3 (Per Socket)	19.25MiB Unified	32MiB Unified
Memory	384GiB	384GiB
Disk	2TiB	2TiB
Operating System	Ubuntu 20.04.6	Ubuntu 22.04.4
Compiler	GCC 11	GCC 11

3.2 Crisis I: The Configuration Game and Rank Inversion

The first fundamental flaw of static workload configurations based evaluation is its extreme susceptibility to configuration nuances, which destroys the objectivity of cross-architecture comparisons. To demonstrate this, we randomly sampled 32 configuration combinations (datasets and concurrency levels) alongside the 1 SPEC-recommended configuration (ref dataset with max logical cores concurrency) for each of the 43 SPEC workloads, resulting in 1,419 unique evaluation configurations.

Fig. 2(a) illustrates the ln-transformed execution time ratio for the 623.xalancbm_s workload across these 33 configurations. The magnitude of the ln-transformed performance ratio fluctuates violently, ranging from -1.274 to +0.245. More alarmingly, a severe **Rank Inversion** occurs: while the SPEC-recommended configuration (SPEC-R, marked as a red triangle point) suggests CPU A

is superior, 31 out of the 32 space-explored configurations clearly demonstrate that CPU B outperforms CPU A.

As shown in the comprehensive boxplots for all 43 workloads (Fig. 2(b)), 28 out of 43 workloads have boxes that straddle the zero reference line (blue dashed line). This proves that under the static workload configurations paradigm, determining which CPU is "better" often devolves into a *Configuration Game*—a mere lottery dependent on which static configuration the benchmark happens to enforce.

3.3 Crisis II: Biased Evaluation Results and Statistical Fragility

A benchmark score is valuable if it is statistically representative of the CPU's general capability on that workload. However, our unlocked SPEC CPU2017 configuration space exposes the systemic **representativeness crisis** inherent in legacy benchmark evaluations.

By analyzing the performance distributions in Fig. 2(b), we observe that the SPEC-recommended configurations (red triangle points) frequently fail to align with the median or mean and even the interquartile range (IQR) of the broader configuration space. In 10 out of 43 workloads, the SPEC-recommended score manifests as a **outlier** residing at the extreme tail of the boxplot. This indicates that traditional static benchmarks rely excessively on specific, empirically chosen configurations. Therefore, the aggregated performance score they report is statistically fragile and fundamentally deviates from the CPU's performance distribution in real-world, dynamic scenarios.

3.4 Crisis III: Architectural Blind Spots

Beyond statistical fragility, static configurations actively obscure the true microarchitectural capabilities of modern processors. To quantify this, we selected the SPECfp speed workloads and explored

a targeted subspace comprising 10 generated datasets, concurrency levels from [1,64], and compiler flags (-O1, -O2, -O3).

By comprehensively traversing this subspace, we discovered optimal configurations that drastically outperform the static SPEC recommendation (-O3 with max logical cores concurrency). Across the evaluated workloads, space-exploration revealed an astonishing average performance improvement of 26.05% for CPU A and 20.85% for CPU B over their respective SPEC baseline scores.

This over 20% performance gap represents a massive **Architectural Blind Spot**. Because legacy benchmarks artificially constrain the execution parameters, they force the CPU to execute sub-optimal microarchitectural paths (e.g., potential cache thrashing induced by a singular dataset size), leaving the processor's peak performance untapped.

In summary, these empirical observations collectively prove that simply patching legacy static workload configurations based on evaluation is insufficient. Modern CPU evaluation necessitates a paradigm shift toward systematic, mathematically rigorous configuration space evaluation.

4 Methodology: The Open Configuration Spaces Paradigm

To fundamentally resolve the three crises of legacy static workload configurations based benchmark (Section 3), BenchCPU pioneers an *Open Configuration Spaces Paradigm*. Rather than relying on deterministic, static workload configurations to proxy CPU performance, BenchCPU redefines benchmarking as a rigorous, distribution-aware stochastic inference process over a configuration space. As illustrated in Fig. 3, our methodology is orchestrated through three systemic pillars: (1) Decoupled Evaluation Space Construction, (2) Momentum-Filtered Stochastic Inference, and (3) Distribution-Aware Result Reporting.

4.1 Decoupled Evaluation Space Construction

Table 2: BenchCPU Workload Distribution and Scale. Each workload (W) is explored across datasets (D), concurrency levels (T), and compiler flags (F).

Lang.	Workload (W)			Data (D)	Thread (T)	Compiler Flag (F)	Total Confgs
	All	Base	Emer.				
C	9	8	1	100	131	3	353.7K
C++	11	9	2	100	131	3	432.3K
Python	15	12	3	100	131	-	196.5K
Java	7	5	2	100	131	-	91.7K
Go	12	8	4	100	131	-	157.2K
Total	54	42	12	-	-	-	1,231.4K

BenchCPU structurally dismantles the monolithic nature of traditional benchmarks by decoupling the evaluation space into three orthogonal, highly extensible dimensions: *Workloads*, *Datasets*, and *Concurrency levels*.

1) The Workload Dimension (The Proxy). Workloads act as proxies for the computational tasks demanded by the real world. To

faithfully capture the migration of real-world computing interests while preserving rigorous longitudinal consistency, we strategically partition workloads into two distinct tiers: *Basic Workloads* and *Emerging Workloads*. Both tiers are constructed using a data-driven Top-N selection methodology to eliminate human bias. We first filter the top 10 programming languages from the TIOBE index [35], selecting C, C++, Python, Java, and Go. Subsequently, we utilize OpenMeter [1] to profile the distribution of open-source projects on GitHub. For *Basic Workloads*, we select the top 100 projects per language to form a candidate set, ensuring a stable baseline that remains immutable for a long-term horizon (e.g., a decade). For *Emerging Workloads*, the candidate set is strictly constrained to projects updated in the year preceding the construction, with the tier updated *annually* to track bleeding-edge industry demands.

The Basic tier anchors the evaluation with industry-standard kernels: (1) *zstd* workload (C) executes high-ratio compression and decompression to stress integer performance and memory bandwidth; and (2) *biogo_igor* workload (Go) implements a bioinformatics pipeline for genomic alignment and serialization to measure CPU efficiency in biological data processing. The Emerging tier reflects the shift toward modern data-intensive services: (1) *transformer_inference* workload (Python) evaluates multi-head self-attention mechanisms and feedforward networks via NLP inference simulation; and (2) *rocksdb* workload (C++) characterizes CPU utilization during high-concurrency key-value manipulation, hashing, and block-level compression.

2) The Dataset and Concurrency Dimensions (The Dynamic Parameters). Unlike legacy suites that stubbornly weld workloads to static datasets (e.g., SPEC's ref dataset), BenchCPU completely opens these evaluation boundaries. We systematically encapsulate both datasets and concurrency levels as dynamically explorable, user-defined runtime parameters, enabling researchers to conduct unbounded architectural space exploration to avoid overlooking "Architectural Blind Spots" (Crisis III). For standardized CPU benchmarking across vendors, as shown in Table 2, BenchCPU defines a *Calibrated Reference Space* within these continuous parameters. Specifically, the dataset size for each workload is parameterized into 100 equal-step configurations within a standardized valid range. Concomitantly, based on the architectural landscape of modern multicore processors, the concurrency dimension provides 131 distinct configurations: spanning the continuous interval of [1, 128], plus heavily-threaded stress points at 200, 256, and 300. By intersecting these parameterized datasets and concurrency levels with our two workload tiers, BenchCPU formally establishes a million-scale *Basic Space* and *Emerging Space*.

4.2 Momentum-Filtered Stochastic Inference

Evaluating the entirety of this million-scale continuous space is computationally impracticable. Conversely, defaulting to a static, hand-picked configuration would instantly regress into the "Configuration Game," "Biased Evaluation Results," and "Architectural Blind Spots" (Three Crises in Section 3). To cost-effectively navigate this combinatorial explosion, BenchCPU treats CPU benchmarking as a rigorous *stochastic inference process*.

Rather than exhaustive execution, BenchCPU automatically publishes periodic (e.g., monthly) benchmark versions by constructing

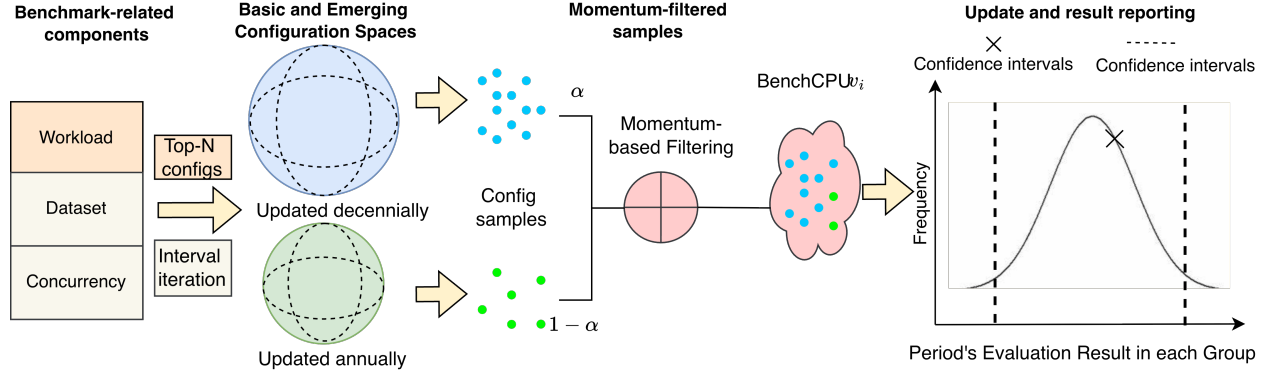


Figure 3: The methodology of BenchCPU.

statistically continuous evaluation sets. We employ *Stratified Random Sampling*, using the individual workload as strata, to randomly select configurations from the overarching evaluation spaces. To maintain cross-version stability and mitigate transient evaluation noise, this sampling process is strictly regulated by a *Momentum-Based Filter* [12]. The configuration for any given BenchCPU version (i) is mathematically defined as a hybrid combination:

$$\text{BenchCPU}^{(i)} = \alpha S_{\text{basic}}^{(i)} + (1 - \alpha) S_{\text{emerging}}^{(i)} \quad (1)$$

where S_{basic} and S_{emerging} denote the sampled configurations from the respective spaces. By setting the hyperparameter α appropriately (e.g., 0.8), the filter guarantees that the baseline evaluation remains anchored in historical consistency while continuously injecting a calibrated degree of emerging computational novelty. Grounded in the Central Limit Theorem (CLT) [9, 20], this sampling mechanism ensures that over multiple versions, the aggregated evaluation results inherently converge to an unbiased estimation of the CPU's true performance across the entire configuration space [9, 10, 22].

4.3 Distribution-Aware Result Reporting

The final pillar of BenchCPU targets the statistical fragility of traditional reporting. We definitively abandon the practice of reporting an aggregated, deterministic "score." Instead, BenchCPU utilizes execution time as the foundational metric and reports performance as a *statistical distribution* over multiple benchmark versions.

For a single evaluated CPU, BenchCPU aggregates its execution times across continuously released configurations and computes the 99% Confidence Interval (CI). This distribution-aware reporting fundamentally proves whether a specific performance outcome is a statistically robust representation of the CPU's capability or merely a biased outlier.

Crucially, for cross-architecture performance comparison (e.g., CPU A vs. CPU B), simply comparing arithmetic means of execution times across different workloads is mathematically flawed and heavily skewed by long-running workloads. To solve this, BenchCPU evaluates the differences in execution times between the identical configurations of the two CPUs using logarithmic transformations.

Specifically, we compute the *ln-difference* of the execution times: $\Delta = \ln(T_A) - \ln(T_B)$. We then report the distribution of these ln-differences alongside its 99% Confidence Interval. Mathematically, the arithmetic mean of ln-differences is strictly equivalent to the logarithm of the geometric mean of their performance ratios:

$$\frac{1}{n} \sum_{i=1}^n (\ln(T_{A,i}) - \ln(T_{B,i})) = \ln \left(\sqrt[n]{\prod_{i=1}^n \frac{T_{A,i}}{T_{B,i}}} \right) \quad (2)$$

This mathematical rigor allows us to properly calculate statistically sound confidence intervals for relative speedups [14]. By reporting this distribution, stakeholders can definitively assess whether CPU A is statistically superior to CPU B across the expansive configuration space, completely neutralizing the evaluation noise introduced by vendor "configuration gaming."

5 Experimental Evaluation

This section evaluates BenchCPU's efficacy in balancing stability, agility, and representativeness. We transition from deterministic "static workload configurations based evaluation" to a stochastic inference methodology, structured around five key objectives:

- **Statistical Robustness:** Evaluating the momentum-based filter's ability to maintain longitudinal consistency during workload transitions.
- **Crisis Resolution:** Empirically validating BenchCPU's immunity to the *Configuration Game* and *Biased Evaluation Results* via multi-dimensional performance distributions and confidence intervals.
- **Dimensional Quantification:** Utilizing ANOVA to mathematically ground our decoupled architecture by quantifying variance contributions from workloads, datasets, and concurrency levels.
- **Microarchitecture Space Coverage:** Employing PCA to show that a minimal BenchCPU subset spans the principal axes of variation captured by SPEC CPU2017 while reaching architectural corners that static suites systematically miss on a typical Intel Xeon Platform.
- **Configuration Scaling:** Extracting *scaling signatures* to distinguish architectural invariants from dataset-driven sensitivities and resolve single-point anomalies.

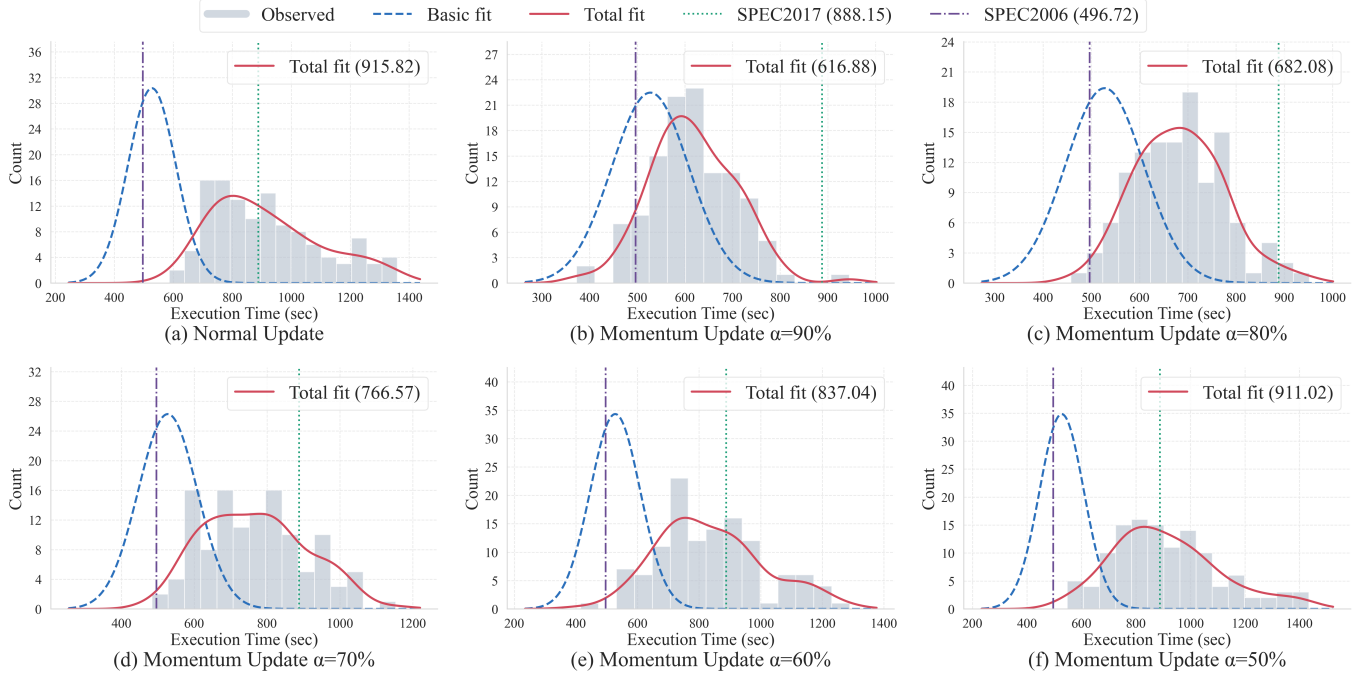


Figure 4: Performance Distribution and Regulated Longitudinal Consistency. Unlike static SPEC snapshots, BenchCPU characterizes performance via stochastic distributions. (a) Naive merge exhibits unmanaged drift and a degraded Gaussian fit. (b)–(f) Momentum filtering ($\alpha = 90\% \rightarrow 50\%$) enables regulated convergence: high α preserves historical fidelity (blue dashed line), while lower α allows a controlled transition to emerging means (red solid line).

Through this framework, we demonstrate that BenchCPU establishes a statistically disciplined paradigm for objective and reproducible CPU evaluation.

5.1 From Static Snapshots to Stochastic Inference: Statistical Robustness

We evaluate BenchCPU’s statistical traits across two processors (Table 1) to demonstrate how its stochastic inference process overcomes the fragility of static suites and ensures longitudinal consistency via momentum filtering.

To capture the performance continuum, we treat benchmarking as a continuous stochastic sampling process across the entire configuration space. This methodology ensures that each evaluation cycle represents a statistically significant derivation from the same underlying performance distribution, establishing a robust framework for longitudinal consistency. By aggregating 120-round stratified samples, as shown in Fig. 4, BenchCPU results converge into a stable normal distribution. This allows performance trends to be tracked as mathematically sound evolutions within a unified operational spectrum, providing a reliable foundation for high-dimensional architectural analysis and long-term performance tracking.[11]

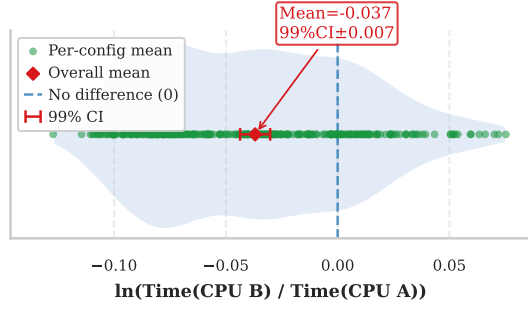
Distributional drift is subsequently mitigated via momentum filtering, addressing the critical challenge of “generational drift” caused by workload updates. As shown in Fig. 4(a), a naive merge of the *Basic* and *Emerging* spaces leads to a significant rightward shift in the performance mean and a noticeable degradation in Gaussian fit. This unmanaged drift represents evaluation volatility,

where the introduction of emerging workloads abruptly obscures the historical baseline, making longitudinal tracking across different suite versions inconsistent. In contrast, the momentum-based filter serves as a regulatory mechanism governing this transition. As depicted in Fig. 4(b)–(f), the filter allows for a regulated convergence toward the new performance mean. Rather than an abrupt jump, BenchCPU enables a smooth evolution of the performance distribution, ensuring that the statistical properties of the benchmark remain under the architect’s control during the update cycle.[27]

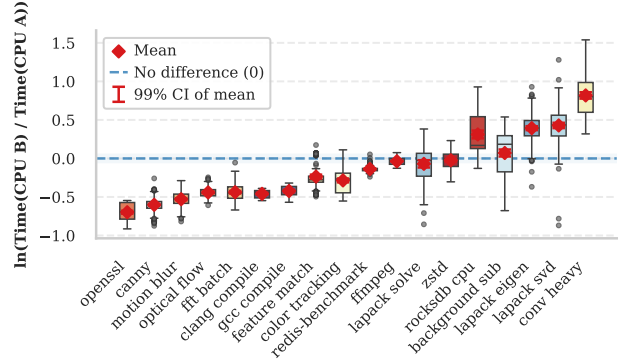
The hyperparameter α further balances baseline stability with market agility, serving as a functional control knob for the system. At high values (e.g., $\alpha = 90\%$), BenchCPU suppresses noise from emerging workloads to preserve a stable, near-normal distribution critical for long-term tracking. As α decreases toward 50%, the influence of the *Emerging Space* grows, eventually converging with the naive merge profile. This tunability allows researchers to determine the pace of baseline evolution, ensuring BenchCPU remains a consistent “anchor” while aggressively tracking bleeding-edge industry demands. We take $\alpha = 80\%$ as the default setting.

5.2 Experimental Evaluation: Resolving the Static Workload Configurations Based Evaluation Crises

To evaluate whether BenchCPU resolves the methodological crises identified in Section 3, we conduct a symmetric space-exploration on BenchCPU’s own workload suite. For each C/C++ workload



(a) *ffmpeg*: distribution of per-configuration ln-ratio



(b) All workloads: ln-ratio of execution times across all configurations

Figure 5: Statistical Stability and Directional Consistency. (a) For *ffmpeg*, the distribution remains strictly consistent (mean = -0.037 , 99% CI = ± 0.007). (b) Across all workloads, BenchCPU's 99% CI of the mean (red diamonds) never crosses the zero reference line, providing a statistically disciplined anchor that resolves Rank Inversion and Biased Evaluation Results.

in BenchCPU, we systematically varied the dataset scale, thread count, and compiler flags, generating hundreds of unique configurations per workload. We then performed the same cross-architecture comparison between CPU A and CPU B.

BenchCPU demonstrates immunity to rank inversion by maintaining directional consistency across the multi-dimensional configuration space. Fig. 5(a) shows that for *ffmpeg*, the performance advantage of CPU B remains an inherent architectural trait (ln-ratio mean -0.037 , CI ± 0.007) rather than a configuration artifact. Unlike SPEC, where 28/43 workloads exhibit distributions straddling the zero reference line (Fig. 2b), all BenchCPU workloads in Fig. 5(b) maintain a 99% CI of the mean that does not cross the zero boundary.[15] This immunity to the *Configuration Game* ensures that stochastic sampling consistently reflects the same architectural ranking, providing objective and reproducible results.

BenchCPU mitigates biased evaluation results by replacing the statistically fragile single score of legacy benchmarks with a statistically robust distribution. In Crisis II (Section 3.3), we observed that some SPEC-recommended configurations manifest as outliers that are deviation from the CPU's true capability. In contrast, BenchCPU's evaluation methodology inherently accounts for configuration-induced variance. By reporting the mean derived from a stochastic sampling process along with its 99% CI, BenchCPU ensures that the evaluation result is an unbiased proxy of the processor's holistic performance. As shown in Fig. 5(b), the tight grouping of configuration points around the median indicates that BenchCPU workloads are designed to be robust across varying scales and parallelism levels. This eliminates the risk of *Biased Evaluation Results*, as the final result is no longer tied to a single, empirically chosen "sweet spot," but is instead anchored by a mathematically sound confidence interval.

5.3 Quantifying Dimensional Contributions: An ANOVA-based Validation

To validate the rationale behind BenchCPU's decoupled architecture, we employ Analysis of Variance (ANOVA) [20] to quantify

Table 3: ANOVA for BenchCPU performance variance (w: workload, d: dataset, t: threads). F-Computed are the computed F-values derived by dividing the component's mean square by the mean square errors. F-Table are the critical F-values obtained via F-table lookup using $p=0.01$ and the corresponding degrees of freedom. F-Computed > F-Table represents statistical significance for the corresponding component.

Component	Var (%)	F-Computed	F-Table
Workload(w)	39.3956%	10445.3027	2.8661
Dataset(d)	16.7369%	17750.3689	4.3512
Thread(t)	20.1568%	21377.3711	4.3512
$w \times d$	5.3972%	1431.0099	2.8661
$w \times t$	12.8934%	3418.5534	2.8661
$d \times t$	3.6934%	3917.0799	4.3512
$w \times d \times t$	1.7079%	452.8241	2.8661
Errors	0.0189%	—	—

the contribution of each orthogonal dimension to the overall performance variance. This experiment aims to provide a rigorous statistical foundation for our shift from static workload configurations based evaluation to performance distribution-aware over open configuration space, ensuring that the dimensions we opened—workload, dataset, and concurrency—are indeed the primary drivers of architectural behavior. To ensure comprehensive coverage, we randomly selected two workloads per programming language (ten levels total), two datasets of significantly different scales to capture input sensitivity, and two concurrency levels (single-thread vs. full-core saturation). The detailed results of the ANOVA and F-test are summarized in Table 3.

The necessity of opening the configuration space is underscored by the diverse factors that drive performance variance. As shown in Table 3, while workload remains the primary variance contributor

(39.40%), concurrency (20.16%) and dataset size (16.74%) exert significant impacts. Crucially, interaction effects (e.g., $w \times t$ at 12.89%) account for a substantial portion of the total variation, proving that CPU performance is not a simple linear summation of components but a complex product of their interactions. By fixing d and t , legacy suites ignore nearly 40% of potential performance variance, creating architectural blind spots where vendor over-optimization thrives.

This multi-dimensional impact finds further statistical justification in our ANOVA results, where the computed F-values (F-Computed) are orders of magnitude higher than the critical F-values (F-Table). The fact that non-workload factors and their interactions contribute as much variance as the workloads themselves reinforces the argument that static workload evaluations are inherently fragile. By systematically exploring these dimensions, BenchCPU captures the multi-dimensional performance continuum required for evaluating modern, non-linear multi-core designs.

5.4 Microarchitecture Space Coverage and Simulation Efficiency

To quantify representativeness and coverage, we project a high-dimensional microarchitecture workload characterization set—spanning instruction mix, frontend dynamics, and memory hierarchy—into a reduced space via PCA. Metrics are collected on a typical Intel Xeon (x86) Platform, and we compute PCA jointly over a pooled population of 1,128 workload instances—1,085 configurations drawn from a pre-release build of BenchCPU together with the 43 SPEC CPU2017 workloads—so that the resulting axes describe a *shared* microarchitectural feature space rather than either suite in isolation. In this joint space, the first two principal components alone account for **47.8%** of the total variance, and the first six reach **81.3%** cumulatively [16, 31].

BenchCPU spans the dominant axes of variation by encompassing the principal PC1–PC2 plane, which accounts for nearly half of the microarchitectural variance and represents the critical dimensions for evaluation representativeness. On this plane, a 5-point BenchCPU subset envelops every SPEC CPU2017 workload, with a mean Spec-to-nearest-Bench distance of 2.03 in the weighted PCA space. The relevant claim here is not that BenchCPU is a strict superset of SPEC, but that it spans the same dominant modes and extends further: BenchCPU’s enclosing region is roughly 4.6× the area of SPEC’s on this plane. This expansion is driven by execution-logic diversification, with FP Ratios up to 0.72 and Branch MPKI across four orders of magnitude, alongside memory-subsystem saturation reaching an L1D MPKI of 158 and an LLC MPKI of 20. These corners are precisely where static suites systematically remain unprobed.

On the lower-energy planes that capture localized rather than systemic behavior, BenchCPU still overlaps with 90.7% of SPEC on PC3–PC4 and 86.05% on PC5–PC6. Crucially, the residual is not diffuse but concentrated on a handful of legacy outliers: on PC5–PC6, for instance, the median Spec-to-nearest-Bench distance is just **0.76**—comparable to that on the principal plane—while a small set of idiosyncratic SPEC workloads (e.g., *mcf*’s pointer-chasing memory access) stretches the maximum to **29.97**, accounting almost entirely for the apparent gap. We therefore view this residual as a property of SPEC’s own outliers rather than a coverage failure of the proposed methodology. By treating datasets and concurrency as

dynamically explorable parameters, BenchCPU exerts orthogonal pressure on the x86 pipeline (e.g., ITLB Miss ≤ 140), yielding a more uniform spread across the general workload distribution than industry standards.

The 5-point subset is identified via D-optimal experimental design [6] to navigate the combinatorial explosion of configurations. This “minimalism with maximum coverage” is critical for RTL-level simulation, where full-suite execution is prohibitive: by transitioning to this statistically disciplined subset, researchers attain comprehensive design-space coverage on x86 targets while reducing simulation turnaround time by orders of magnitude.

5.5 Exploration of the Configuration Space: Microarchitecture Workload Characterization

Beyond the workload axis, BenchCPU exposes a continuous input-scale axis that admits dense, uniform sampling rather than the conventional small/medium/large triage. We exploit this axis to extract *scaling signatures*—per-workload trajectories of microarchitectural counters as a function of data volume, which fingerprint how each workload stresses the memory hierarchy. On Kunpeng-920, we sweep two representative workloads (*cassandra*, *transformer_inference*) over 100 contiguous scales and visualize the results on a log-scale MPKI axis (Fig. 6). We show that these signatures (i) separate workloads into qualitatively distinct scaling regimes that coarse sampling cannot resolve, and (ii) enable a causal decomposition of counter-intuitive metrics that no single operating point can disambiguate.

Fig. 6 exposes two scaling regimes that reflect how each workload’s locality relates to its input-size parameter. *cassandra* (panels (a,b)) exhibits *scaling invariance*: all four MPKI curves remain flat across 100 scales (Coefficient of Variation $\sigma/\mu < 5\%$ per metric) despite relatively high absolute values. This invariance is workload-intrinsic: each operation traverses a fixed server-side pipeline (bloom-filter probe, memtable / SSTable index lookup, row deserialization on the Cassandra stack) whose per-operation locality is set by the runtime and data structures rather than by request count. Sweeping the input scales the operation count but not the work per operation, so the instruction-normalized MPKI is invariant by construction, driven by the JIT-compiled code footprint and pointer-chasing rather than data volume. *transformer_inference* (panel (c)), by contrast, exhibits *scaling volatility* ($\sigma/\mu \approx 27\%$): its input parameter directly controls tensor dimensions, so each sweep point realizes a different per-operation memory footprint. The trajectory exhibits sharp microarchitectural singularities at Scale 8—a canonical example of behavior that a coarse small/medium/large sampling would miss or misclassify as noise, and that dense sampling elevates to a first-class diagnostic. The same invariance/volatility split reproduces on our Intel Xeon (x86) Platform, indicating that the taxonomy captures a *workload-structural* property rather than a machine- or ISA-specific artifact. These two regimes—*runtime-bounded* and *data-bounded* footprints—represent complementary and orthogonal axes of workload characterization that are both worth capturing.

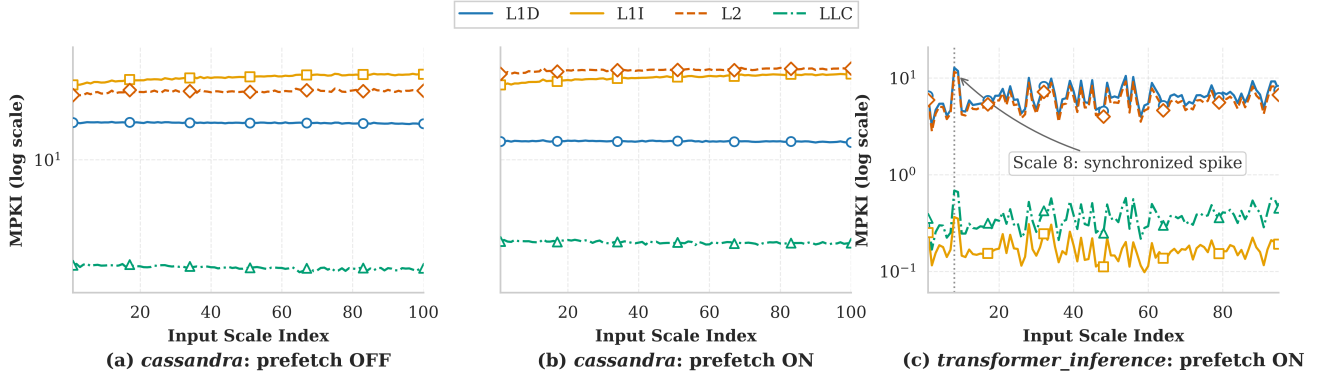


Figure 6: Scaling signatures (Kunpeng-920). Log-scale MPKI across 100 input scales. *cassandra* (a, b) is scale-invariant; L1I is unchanged between prefetcher states while L2 rises by ~ 13 MPKI with prefetch on, separating structural and dynamic contributions to the apparent L2 inversion. *transformer_inference* (c) is scale-volatile, with a synchronized cache spike at Scale 8 and near-zero L1I MPKI.

The log-scale view further reveals consistent topological parallelism among the L1D, L2, and LLC curves, implying a scale-invariant filtering ratio between adjacent levels (the fraction of refills propagating from level i to $i+1$, e.g., $L2_{\text{refill}}/(L1D_{\text{refill}}+L1I_{\text{refill}})$ at the $L1 \rightarrow L2$ boundary). A striking observation is that *cassandra*'s L2 refill MPKI exceeds its L1D MPKI by $3.25\times$ (panel (b))—a ratio that, reported at any fixed-dataset workload, would naturally be attributed to an aggressive L2 prefetcher. Sweeping read-n across 100 points under both prefetch-on and prefetch-off BIOS settings (a 2×100 natural experiment; panels (a,b)) refutes this static workload configurations based evaluation and decomposes the inversion into two *independent, additive* mechanisms, each identifiable by a distinct signature along the input axis. **(i) Microarchitecture mechanism.** *cassandra*'s L1I MPKI averages ≈ 39 —three times its L1D MPKI—with $\sigma/\mu < 5\%$ across all 100 scales, and its two prefetcher-state means differ by only 0.26% (38.98 vs. 38.88). The AArch64-unified-L2 PMU specification folds these instruction-cache refills into L2D_CACHE_REFILL, invisible to L1D counters; this component alone establishes a $1.68\times$ L2/L1D MPKI ratio even with the prefetcher *disabled* (panel (a)), confirming that the inversion's baseline arises from *deterministic unified-L2 accounting* rather than from speculative prefetch traffic. **(ii) Prefetch behavior.** Enabling the prefetcher (panel (b)) injects a *near-constant* +13 MPKI of L2-to-memory refills across the entire sweep while reducing L1D demand misses by only ~ 5 MPKI, a bandwidth-for-latency exchange rate of $2.7\times^1$ extra memory transactions per demand miss avoided—the signature of a pointer-chasing access pattern only partially captured by the stride prefetcher. Subtracting this speculative component, the demand-filter ratio $L2/(L1D+L1I)$ stabilizes at ≈ 0.55 with less than 10% variation across all input scales and prefetcher states, identifying L2 filtering efficiency as an *architectural* rather than a workload property. *transformer_inference* (panel (c)), by contrast, exhibits $L1 \approx L2$ not because the prefetcher is suppressed but because its tight GEMM kernels yield an L1I MPKI below 0.2—the structural driver of *cassandra*'s inversion is simply absent; its

synchronized spikes therefore reflect capacity-driven overflow as streaming tensor footprints traverse successive cache boundaries, not conflict-induced pathology [21, 24]. Neither mechanism is recoverable from static workload configurations based evaluation: the two are observationally equivalent in isolation and separable only once the dataset dimension is densely sampled.

Taken together, these case studies position BenchCPU not merely as a workload corpus but as a probe that separates *structural* microarchitectural effects—unified-L2 accounting, instruction-cache pressure from managed runtimes, architectural-dependent scale-invariant filter ratios—from *dynamic* ones—prefetcher-injected traffic, capacity-induced regime transitions. This separation is infeasible at a single operating point and is the central methodological payoff of exposing the dataset dimension: claims such as "the L2 is a bottleneck" or "the prefetcher is too aggressive" acquire testable, quantitative signatures that architects, runtime implementers, and benchmark designers can act upon during early-stage design-space exploration.

6 Conclusion

The paradigm of evaluating modern processors—characterized by high complexity and non-linear performance scaling—through static workload configurations is increasingly inadequate. To address the inherent limitations in current rigorous CPU evaluation, we presented BenchCPU, a performance distribution-aware CPU benchmarking over open configuration spaces. BenchCPU decouples the evaluation space into multiple orthogonal, highly extensible dimensions, including but not limited to workload, dataset, and concurrency level, and constructs a highly stable Basic Space alongside a dynamically evolving Emerging Space. To navigate the combinatorial explosion, BenchCPU employs stratified random sampling and a momentum-based filter, guaranteeing cost-effective, mathematically rigorous, and longitudinally consistent performance distributions. Extensive evaluations demonstrate BenchCPU's superiority in solving the inherent limitations of static workload configurations based benchmarks and exploring the configuration space to gain deep architectural insights.

¹Computed as $\Delta L2 \text{ MPKI} / |\Delta L1D \text{ MPKI}| = 13.18/4.82$, both measured as the difference of 100-scale means between prefetcher states.

References

- [1] BenchCouncil. 2024. OpenMeter. <https://openmeter.benchcouncil.org>.
- [2] Lawrence Benson, Carsten Binnig, Jan-Micha Bodensohn, Federico Lorenzi, Jigao Luo, Danica Porobic, Tilmann Rabl, Anupam Sanghi, Russell Sears, Pinar Tözün, et al. 2024. Surprise Benchmarking: The Why, What, and How. In *Proceedings of the Tenth International Workshop on Testing Database Systems*. 1–8.
- [3] Christian Bienia, Sanjeev Kumar, and Kai Li. 2008. PARSEC vs. SPLASH-2: A quantitative comparison of two multithreaded benchmark suites on Chip-Multiprocessors. (2008), 47–56.
- [4] C. Bienia, S. Kumar, J.P. Singh, and K. Li. 2008. The PARSEC benchmark suite: Characterization and architectural implications. In *Proceedings of the 17th international conference on Parallel architectures and compilation techniques*. ACM, 72–81.
- [5] Simone Campanoni, Giovanni Agosta, Stefano Crespi Reghizzi, and Andrea Di Biagio. 2010. A highly flexible, parallel virtual machine: Design and experience of ILDJIT. *Software: Practice and Experience* 40, 2 (2010), 177–207.
- [6] P Fernandes de Aguiar, B Bourguignon, MS Khots, DL Massart, and R Phan-Than-Luu. 1995. D-optimal designs. *Chemometrics and intelligent laboratory systems* 30, 2 (1995), 199–210.
- [7] Kamran Farhadi, Kai Yi Kenneth Po, and Levon Stepanian. 2003. Trace Based Optimizations of the Jupiter JVM Using DynamoRIO. (2003).
- [8] M. Ferdman, A. Adileh, O. Kocerber, S. Volos, M. Alisafae, D. Jevdjic, C. Kaynak, A. Popescu, A. Ailamaki, and B. Falsafi. 2012. Clearing the Clouds: A Study of Emerging Workloads on Modern Hardware. *Architectural Support for Programming Languages and Operating Systems* (2012).
- [9] Ronald Aylmer Fisher. 1970. Statistical methods for research workers. In *Breakthroughs in statistics: Methodology and distribution*. Springer, 66–70.
- [10] Ronald Aylmer Fisher and Ronald A Fisher. 1971. *The design of experiments*. Springer.
- [11] Andy Georges, Dries Buytaert, and Lieven Eeckhout. 2007. Statistically rigorous java performance evaluation. *ACM SIGPLAN Notices* 42, 10 (2007), 57–76.
- [12] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. 2020. Momentum contrast for unsupervised visual representation learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 9729–9738.
- [13] John L Hennessy and David A Patterson. 2019. A new golden age for computer architecture. *Commun. ACM* 62, 2 (2019), 48–60.
- [14] John L. Hennessy and David A. Patterson. 2011. *Computer Architecture, Fifth Edition: A Quantitative Approach* (5th ed.). Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.
- [15] Torsten Hoefler and Roberto Belli. 2015. Scientific benchmarking of parallel computing systems: twelve ways to tell the masses when reporting performance results. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*. 1–12.
- [16] Kenneth Hoste and Lieven Eeckhout. 2007. Microarchitecture-independent workload characterization. *IEEE micro* 27, 3 (2007), 63–72.
- [17] Kenneth Hoste and Lieven Eeckhout. 2006. Comparing Benchmarks Using Key Microarchitecture-Independent Characteristics. (2006), 83–92.
- [18] Kenneth Hoste, Lieven Eeckhout, and Hendrik Blockeel. 2007. Analyzing commercial processor performance numbers for predicting performance of applications of interest. 35, 1 (2007), 375–376.
- [19] Kenneth Hoste, Aashish Phansalkar, Lieven Eeckhout, Andy Georges, Lizy K John, and Koen De Bossche. 2006. Performance prediction based on inherent program similarity. (2006), 114–122.
- [20] Raj Jain. 1991. *The art of computer systems performance analysis*. Vol. 182. John Wiley & Sons Chichester.
- [21] A. Jaleel, M. Mattina, and B. Jacob. 2006. Last level cache (LLC) performance of data mining workloads on a CMP—a case study of parallel bioinformatics workloads. In *High-Performance Computer Architecture, 2006. The Twelfth International Symposium on*. IEEE, 88–98.
- [22] Roger E Kirk. 2009. Experimental design. *Sage handbook of quantitative methods in psychology* (2009), 23–45.
- [23] Tuomas Koskela, Ilektra Christidi, Mosè Giordano, Emily Dubrovskaya, Jamie Quinn, Christopher Maynard, Dave Case, Kaan Olgu, and Tom Deakin. 2023. Principles for automated and reproducible benchmarking. In *Proceedings of the SC'23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*. 609–618.
- [24] Dusol Lee, Inhyuk Choi, Chanyoung Lee, Sungjin Lee, and Jihong Kim. 2023. P2cache: An application-directed page cache for improving performance of data-intensive applications. In *Proceedings of the 15th ACM Workshop on Hot Topics in Storage and File Systems*. 31–36.
- [25] Ankur Limaye and Tosiron Adegbiya. 2018. A workload characterization of the spec cpu2017 benchmark suite. In *2018 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 149–158.
- [26] David S Moore and Stephane Kirkland. 2007. *The basic practice of statistics*. Vol. 2. WH Freeman New York.
- [27] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F Sweeney. 2009. Producing wrong data without doing anything obviously wrong! *ACM Sigplan Notices* 44, 3 (2009), 265–276.
- [28] Reena Panda, Shuang Song, Joseph Dean, and Lizy K John. 2018. Wait of a Decade: Did SPEC CPU 2017 Broaden the Performance Horizon? (2018), 271–282.
- [29] D.A. Patterson and J.L. Hennessy. 2009. *Computer organization and design: the hardware/software interface*. Morgan Kaufmann.
- [30] Aashish Phansalkar, Ajay Joshi, Lieven Eeckhout, and Lizy Kurian John. 2005. Measuring program similarity: Experiments with SPEC CPU benchmark suites. In *IEEE International Symposium on Performance Analysis of Systems and Software, 2005. ISPASS 2005*. IEEE, 10–20.
- [31] A. Phansalkar, A. Joshi, and L.K. John. 2007. Analysis of redundancy and application balance in the SPEC CPU2006 benchmark suite. *ACM SIGARCH Computer Architecture News* 35, 2 (2007), 412–423.
- [32] Yakun Sophia Shao and David Brooks. 2013. ISA-independent workload characterization and its implications for specialized architectures. In *2013 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 245–255. doi:10.1109/ISPASS.2013.6557175
- [33] SPEC. 2017. SPEC CPU Results. Available at <https://www.spec.org/cgi-bin/osgresults>.
- [34] SPEC. 2025. SPEC CPU. Available at <https://www.spec.org/cpu>.
- [35] TIOBE. 2026. TIOBE index. <https://www.tiobe.com/tiobe-index>.
- [36] Chenxi Wang, Lei Wang, Wanling Gao, Yikang Yang, Yutong Zhou, and Jianfeng Zhan. 2024. Achieving consistent and comparable CPU evaluation outcomes. *arXiv preprint arXiv:2411.08494* (2024).
- [37] Lei Wang, Kaiyong Yang, Chenxi Wang, Wanling Gao, Chunjie Luo, Fan Zhang, Zhongxin Ge, Li Zhang, Guoxin Kang, and Jianfeng Zhan. 2023. WPC: Whole-picture Workload Characterization. *arXiv preprint arXiv:2302.12954* (2023).
- [38] Yang Wang, Miao Yu, Yujie Hui, Fang Zhou, Yuyang Huang, Rui Zhu, Xueyuan Ren, Tianxi Li, and Xiaoyi Lu. 2022. A study of database performance sensitivity to experiment settings. *Proceedings of the VLDB Endowment* 15, 7 (2022).
- [39] David F Williamson, Robert A Parker, and Juliette S Kendrick. 1989. The box plot: a simple visual method to interpret data. *Annals of internal medicine* 110, 11 (1989), 916–921.
- [40] Steven Cameron Woo, Moriyoshi Ohara, Evan Torrie, Jaswinder Pal Singh, and Anoop Gupta. 1995. The SPLASH-2 programs: characterization and methodological considerations. 23, 2 (1995), 24–36.
- [41] Ahmad Yasin, Yosi Ben-Asher, and Avi Mendelson. 2014. Deep-dive analysis of the data analytics workload in cloudsuite. In *2014 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 202–211.
- [42] Jianfeng Zhan. 2025. Fundamental concepts and methodologies in evaluatology. 100188 pages.
- [43] Jianfeng Zhan. 2024. A short summary of evaluatology: The science and engineering of evaluation. *BenchCouncil Transactions on Benchmarks, Standards and Evaluations* (2024), 100175.
- [44] Jianfeng Zhan, Lei Wang, Wanling Gao, Hongxiao Li, Chenxi Wang, Yunyou Huang, Yatao Li, Zhengxin Yang, Guoxin Kang, Chunjie Luo, Hainan Ye, Shaopeng Dai, and Zhifei Zhang. 2024. Evaluatology: The science and engineering of evaluation. *BenchCouncil Transactions on Benchmarks, Standards and Evaluations* 4, 1 (2024), 100162.
- [45] Jianfeng Zhan, Lei Wang, Wanling Gao, Chenxi Wang, Hongxiao Li, Fanda Fan, and Guoxin Kang. 2025. *Evaluatology: The Science of Uncovering the Effects* (version 1.0 (first edition, 2025) ed.). BenchCouncil Press, Hong Kong.
- [46] Xusheng Zhan, Yungang Bao, Christian Bienia, and Kai Li. 2017. PARSEC3.0: A Multicore Benchmark Suite with Network Stacks and SPLASH-2X. *ACM Sigarch Computer Architecture News* 44, 5 (2017), 1–16.